

Microservices Communication Patterns

Complete guide to communication strategies between microservices.

Communication Types

1. Synchronous Communication

Direct service-to-service calls with immediate response

REST API (HTTP/HTTPS)

When to use:

- Simple CRUD operations
- Public APIs
- Request-response patterns

Example:

```
import requests

# Service A calls Service B
response = requests.post(
    'http://service-b:8080/api/orders',
    json={'user_id': 123, 'product_id': 456},
    timeout=5
)

if response.status_code == 200:
    order = response.json()
```

Pros:

- Simple to implement
- Easy to debug
- Wide tooling support

Cons:

- Tight coupling
- Cascading failures
- Latency accumulation

gRPC

When to use:

- High-performance requirements
- Internal microservices (not public)
- Strong typing needed

Example:

```
import grpc
import order_pb2
import order_pb2_grpc

# Service A calls Service B via gRPC
channel = grpc.insecure_channel('service-b:50051')
stub = order_pb2_grpc.OrderServiceStub(channel)

request = order_pb2.CreateOrderRequest(
    user_id=123,
    product_id=456
)

response = stub.CreateOrder(request, timeout=5)
```

Pros:

- 7x faster than REST
- Binary protocol (smaller payload)
- Built-in streaming
- Strong contracts (protobuf)

Cons:

- Harder debugging
- Less human-readable
- Limited browser support

GraphQL**When to use:**

- Mobile/frontend needs flexible queries
- Aggregating multiple services
- Avoiding over-fetching

Example:

```
from graphql import GraphQLObjectType, GraphQLField, GraphQLString

# Gateway aggregates multiple services
schema = GraphQLObjectType(
    'Query',
```

```
lambda: {  
    'user': GraphQLField(UserType, resolve=fetch_from_user_service),  
    'orders': GraphQLField(OrderType, resolve=fetch_from_order_service)  
}  
)
```

Pros:

- Single endpoint
- Client-driven queries
- Reduces over-fetching

Cons:

- Complex caching
- N+1 query problem
- Learning curve

2. Asynchronous Communication

Decoupled messaging without waiting for response**Message Queue (Point-to-Point)****When to use:**

- Job processing
- Load leveling
- Guaranteed delivery needed

Example with RabbitMQ:

```
import pika  
  
# Producer (Service A)  
connection = pika.BlockingConnection(  
    pika.ConnectionParameters('rabbitmq')  
)  
channel = connection.channel()  
channel.queue_declare(queue='orders', durable=True)  
  
channel.basic_publish(  
    exchange='',  
    routing_key='orders',  
    body='{"order_id": 123}',  
    properties=pika.BasicProperties(delivery_mode=2)  
)  
  
# Consumer (Service B)  
def callback(ch, method, properties, body):  
    print(f"Processing order: {body}")
```

```
# Process order
ch.basic_ack(delivery_tag=method.delivery_tag)

channel.basic_consume(queue='orders', on_message_callback=callback)
channel.start_consuming()
```

Pros:

- Decouples services
- Load balancing
- Retry logic built-in

Cons:

- No immediate response
- Debugging harder
- Message order not guaranteed

Event Bus (Pub/Sub)**When to use:**

- Multiple services need same event
- Event-driven architecture
- Broadcasting updates

Example with Kafka:

```
from kafka import KafkaProducer, KafkaConsumer
import json

# Publisher (Service A)
producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

producer.send('order-created', {
    'order_id': 123,
    'user_id': 456,
    'timestamp': '2026-08-05T10:00:00Z'
})

# Subscriber (Service B, C, D)
consumer = KafkaConsumer(
    'order-created',
    bootstrap_servers=['kafka:9092'],
    group_id='inventory-service',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)
```

```
for message in consumer:
    event = message.value
    print(f"Order created: {event['order_id']}")
    # Update inventory
```

Pros:

- True decoupling
- Multiple subscribers
- Event replay capability
- Scales horizontally

Cons:

- Eventual consistency
- Complex debugging
- Schema evolution needed

Event Sourcing**When to use:**

- Audit trail required
- Time-travel debugging
- Complex domain logic

Example:

```
# Store events instead of state
events = [
    {'type': 'OrderCreated', 'order_id': 123, 'user_id': 456},
    {'type': 'PaymentProcessed', 'order_id': 123, 'amount': 99.99},
    {'type': 'OrderShipped', 'order_id': 123, 'tracking': 'ABC123'}
]

# Rebuild state from events
def rebuild_order_state(order_id):
    state = {}
    for event in get_events(order_id):
        if event['type'] == 'OrderCreated':
            state['status'] = 'created'
            state['user_id'] = event['user_id']
        elif event['type'] == 'PaymentProcessed':
            state['status'] = 'paid'
            state['amount'] = event['amount']
        elif event['type'] == 'OrderShipped':
            state['status'] = 'shipped'
            state['tracking'] = event['tracking']
    return state
```

Pros:

- Complete audit trail
- Can rebuild any state
- Temporal queries

Cons:

- Storage overhead
- Query complexity
- Schema evolution hard

Communication Patterns

1. API Gateway Pattern

Single entry point for all clients

```
from fastapi import FastAPI
import httpx

app = FastAPI()

@app.get("/api/user/{user_id}/dashboard")
async def get_dashboard(user_id: int):
    async with httpx.AsyncClient() as client:
        # Aggregate multiple services
        user = await client.get(f"http://user-service/users/{user_id}")
        orders = await client.get(f"http://order-service/orders?user={user_id}")
        recommendations = await client.get(f"http://rec-
service/recommend/{user_id}")

    return {
        "user": user.json(),
        "orders": orders.json(),
        "recommendations": recommendations.json()
    }
```

Pros:

- Single entry point
- Authentication/authorization centralized
- Request aggregation
- Protocol translation

Cons:

- Single point of failure
- Can become bottleneck
- Additional latency

2. Service Mesh Pattern

Infrastructure layer for service-to-service communication

```
# Istio example
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: order-service
spec:
  hosts:
  - order-service
  http:
  - match:
    - headers:
        user-type:
          exact: premium
    route:
    - destination:
        host: order-service
        subset: v2
        weight: 100
    - route:
    - destination:
        host: order-service
        subset: v1
        weight: 100
```

Features:

- Load balancing
- Circuit breaking
- Retries
- Timeouts
- mTLS encryption
- Distributed tracing

Pros:

- Traffic management
- Security built-in
- Observability
- Language agnostic

Cons:

- Complex setup
- Resource overhead
- Learning curve

3. Backend for Frontend (BFF)

Dedicated backend for each client type

```
# Mobile BFF
@app.get("/mobile/api/home")
async def mobile_home(user_id: int):
    # Minimal data for mobile
    return {
        "user": get_user_summary(user_id),
        "top_orders": get_recent_orders(user_id, limit=5)
    }

# Web BFF
@app.get("/web/api/home")
async def web_home(user_id: int):
    # Rich data for web
    return {
        "user": get_user_full(user_id),
        "orders": get_recent_orders(user_id, limit=20),
        "recommendations": get_recommendations(user_id),
        "analytics": get_user_analytics(user_id)
    }
```

Pros:

- Optimized per client
- Independent evolution
- Reduced over-fetching

Cons:

- Code duplication
- More services to maintain

4. Saga Pattern

Distributed transactions across services

Choreography (Event-driven):

```
# Order Service
def create_order(order):
    save_order(order)
    publish_event('OrderCreated', order)

# Payment Service listens to OrderCreated
def handle_order_created(event):
    result = process_payment(event.order_id)
    if result.success:
```



```
        publish_event('PaymentSucceeded', event.order_id)
    else:
        publish_event('PaymentFailed', event.order_id)

# Inventory Service listens to PaymentSucceeded
def handle_payment_succeeded(event):
    reserve_inventory(event.order_id)
    publish_event('InventoryReserved', event.order_id)

# Order Service listens to InventoryReserved
def handle_inventory_reserved(event):
    update_order_status(event.order_id, 'confirmed')
```

Orchestration (Centralized):

```
# Order Orchestrator
class OrderSaga:
    def execute(self, order):
        # Step 1: Create order
        order_id = order_service.create(order)

        try:
            # Step 2: Process payment
            payment_service.charge(order_id)

            # Step 3: Reserve inventory
            inventory_service.reserve(order_id)

            # Step 4: Confirm order
            order_service.confirm(order_id)

        except PaymentFailed:
            order_service.cancel(order_id)
        except InventoryUnavailable:
            payment_service.refund(order_id)
            order_service.cancel(order_id)
```

Pros:

- Distributed transaction handling
- Compensating actions
- Clear business logic

Cons:

- Complex implementation
- Debugging difficult
- Latency increases

Service Discovery

Static Configuration

```
# config.py
SERVICES = {
    'user-service': 'http://user-service:8080',
    'order-service': 'http://order-service:8080'
}
```

Pros: Simple, predictable **Cons:** Manual updates, no auto-scaling

DNS-Based Discovery

```
import requests

# Kubernetes DNS
response = requests.get('http://order-service.default.svc.cluster.local/orders')
```

Pros: Built-in to K8s, no additional service **Cons:** DNS caching issues, no health checks

Service Registry (Consul, Eureka)

```
import consul

# Register service
c = consul.Consul()
c.agent.service.register(
    'order-service',
    service_id='order-service-1',
    address='10.0.0.5',
    port=8080,
    check=consul.Check.http('http://10.0.0.5:8080/health', '10s')
)

# Discover service
services = c.health.service('order-service', passing=True)
endpoint = f"http://{services[0]['Service']['Address']}:{services[0]['Service']
['Port']}"
```

Pros: Dynamic registration, health checks, load balancing **Cons:** Additional infrastructure, complexity

Best Practices

1. Circuit Breaker

Prevent cascading failures

```
from pybreaker import CircuitBreaker

breaker = CircuitBreaker(
    fail_max=5,
    timeout_duration=60
)

@breaker
def call_external_service():
    return requests.get('http://external-service/api')

try:
    result = call_external_service()
except CircuitBreakerError:
    return fallback_response()
```

2. Retry with Exponential Backoff

```
import time
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=2, max=60)
)
def call_with_retry():
    return requests.get('http://service/api', timeout=5)
```

3. Timeout Configuration

```
# Set aggressive timeouts
response = requests.get(
    'http://service/api',
    timeout=(2, 5) # (connect_timeout, read_timeout)
)
```

4. Idempotency

Make operations safe to retry

```
@app.post("/orders")
def create_order(request: OrderRequest, idempotency_key: str):
    # Check if already processed
    existing = get_order_by_idempotency_key(idempotency_key)
    if existing:
        return existing
```

```
# Process new order
order = create_new_order(request)
save_idempotency_key(idempotency_key, order.id)
return order
```

5. Rate Limiting

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@app.post("/orders")
@limiter.limit("100/minute")
def create_order():
    pass
```

Comparison Matrix

Pattern	Latency	Coupling	Reliability	Complexity	Use Case
REST API	Low	High	Medium	Low	Simple CRUD
gRPC	Very Low	High	High	Medium	High-performance
Message Queue	High	Low	High	Medium	Async jobs
Event Bus	High	Very Low	High	High	Event-driven
GraphQL	Medium	Medium	Medium	High	Flexible queries
Service Mesh	Medium	Low	Very High	Very High	Enterprise scale

Interview Tips

Common questions:

1. "How do you handle failures?"
 - Circuit breakers, retries, timeouts
 - Fallback responses
 - Graceful degradation
2. "Sync vs Async communication?"
 - Sync: Simple, immediate response, tight coupling
 - Async: Scalable, decoupled, eventual consistency
3. "How to ensure data consistency?"

- Saga pattern for distributed transactions
- Event sourcing for audit trail
- Two-phase commit (avoid if possible)

4. "How to discover services?"

- Service registry (Consul, Eureka)
- DNS-based (Kubernetes)
- API Gateway

5. "How to prevent cascading failures?"

- Circuit breakers
- Bulkheads (isolate resources)
- Rate limiting
- Timeouts

Real-World Example

E-commerce Order Flow:

```
# 1. API Gateway receives request
@gateway.post("/orders")
async def create_order(order: OrderRequest):

    # 2. Validate with User Service (REST)
    user = await http_client.get(f"http://user-service/users/{order.user_id}")

    # 3. Check inventory (gRPC for speed)
    inventory = await grpc_client.CheckInventory(order.product_id)

    if not inventory.available:
        return {"error": "Out of stock"}

    # 4. Create order (synchronous)
    order = await http_client.post("http://order-service/orders", order)

    # 5. Publish event for async processing (Kafka)
    await kafka_producer.send('order-created', {
        'order_id': order.id,
        'user_id': order.user_id,
        'product_id': order.product_id
    })

    return order

# Payment Service subscribes to order-created
@kafka_consumer.subscribe('order-created')
async def process_payment(event):
    payment = charge_customer(event['user_id'], event['order_id'])

    if payment.success:
```

```
        await kafka_producer.send('payment-succeeded', event)
    else:
        await kafka_producer.send('payment-failed', event)

# Inventory Service subscribes to payment-succeeded
@kafka_consumer.subscribe('payment-succeeded')
async def reserve_inventory(event):
    reserve(event['product_id'], event['order_id'])
    await kafka_producer.send('inventory-reserved', event)

# Notification Service subscribes to inventory-reserved
@kafka_consumer.subscribe('inventory-reserved')
async def send_confirmation(event):
    send_email(event['user_id'], event['order_id'])
```

Tools & Technologies

Message Brokers:

- RabbitMQ (traditional queue)
- Apache Kafka (event streaming)
- AWS SQS (managed queue)
- Redis Streams (lightweight)

Service Mesh:

- Istio
- Linkerd
- Consul Connect

API Gateways:

- Kong
- AWS API Gateway
- Nginx
- Traefik

Service Discovery:

- Consul
- Eureka
- etcd
- Zookeeper